Heterogeneidade de Modelos e Consumo de Bateria em Aprendizado Federado

Revisão de trabalhos, sugestões de pesquisa e roteiro de implementação no projeto AprendFed

18 de outubro de 2026

> **Observação.** Este documento reúne uma conversa com o Claude Code. Dos artigos citados, apenas o LeanFed foi lido na íntegra; os demais foram avaliados pelos resumos e trechos encontrados na busca. Antes de fechar o tema, leia por completo o DR-FL e o artigo do IEEE TCCN, que são os mais próximos da proposta, e discuta com o orientador.

Conteúdo

- 1. Trabalhos relacionados
- 2. Como o LeanFed modela a bateria
- 3. Sugestões para o mestrado
- 4. Roteiro de implementação (Passos 1 a 3)
- 5. Referências

1. Trabalhos relacionados

A interseção entre heterogeneidade de modelos e consumo de bateria é pouco explorada. A maior parte da literatura trata ou de modelos heterogêneos (submodelos de tamanhos diferentes por dispositivo), ou de bateria e energia (seleção de clientes, carga de trabalho), mas raramente das duas coisas juntas.

1.1 Trabalhos que combinam modelos heterogêneos e energia/bateria

DR-FL (Xia & Shi, 2024) — arXiv 2405.08183

É o mais próximo do tema. Usa aprendizado por reforço multiagente (MARL) para escolher, a cada rodada, quais clientes participam e qual modelo heterogêneo cada um treina, de acordo com a capacidade de computação e de energia do dispositivo. O cenário é de dispositivos com bateria. O argumento central é que o desperdício de energia no FL tradicional vem do descompasso entre dispositivos heterogêneos e um modelo único para todos.

Seleção adaptativa de modelos para agentes móveis (Zhang et al., IEEE TCCN) — PDF

Parte do problema de clientes que abandonam o treinamento quando a bateria acaba. Aponta que trabalhos anteriores atribuíam modelos heterogêneos para economizar computação, mas ignoravam que a capacidade do dispositivo, incluindo a bateria, muda ao longo do tempo. *(Autoria conforme o resumo da busca; confirme no PDF.)*

Pfeiffer et al. (2024) — Energy-Aware Heterogeneous FL via Approximate Systolic DNN Accelerators — arXiv 2402.18569

Trata dispositivos com energia disponível diferente. Em vez de dar submodelos menores aos clientes, usa hardware aproximado. Os autores criticam as abordagens de submodelo, dizendo que perdem precisão — uma crítica que vale testar na sua dissertação.

1.2 Trabalhos vizinhos

Tratam energia sem modelos heterogêneos, ou o contrário.

- **EAFL** (Arouj & Abdelmoniem, FedEdge/MobiCom 2022, arXiv 2208.04505): escolhe clientes pela bateria restante, equilibrando tempo até a precisão-alvo e bateria que sobra nos dispositivos. Todos treinam o mesmo modelo. Boa base de comparação.
- **AutoFL** (Kim & Wu, MICRO 2021, arXiv 2107.08147): aprendizado por reforço para escolher os participantes e onde cada um executa o treino (CPU, GPU…), visando eficiência energética. Modelo único.
- **LeanFed** (Pereira et al., arXiv 2412.02289): ajusta a fração de dados que cada cliente usa no treino para que a bateria dure todas as rodadas. Detalhado na seção 2.
- **Seleção dinâmica de modelos** (arXiv 2409.08858): atribui modelos de tamanhos diferentes a cada dispositivo, mas não modela energia explicitamente (pelo trecho consultado).
- **Heterogeneidade de modelos "pura"** (citados de memória): HeteroFL (ICLR 2021), FjORD (NeurIPS 2021), FedRolex (NeurIPS 2022), ScaleFL (CVPR 2023) e DepthFL (ICLR 2023). Variam largura ou profundidade do modelo por dispositivo, mas medem custo em FLOPs ou memória, não em bateria.

1.3 Lacuna aparente

A maioria dos métodos de submodelos (HeteroFL, FjORD) escolhe o tamanho do modelo **uma vez**, por categoria de dispositivo. Escolher a largura **a cada rodada conforme a bateria restante**, levando em conta a descarga e o abandono de clientes, só aparece no DR-FL e no artigo do TCCN. Há espaço para contribuir aí.

2. Como o LeanFed modela a bateria

> **Resposta curta.** A bateria é **totalmente simulada**. Não há medição em bateria nem em dispositivo real: os experimentos rodaram em PyTorch numa única GPU RTX 3090, e a "bateria" é um número que diminui a cada época de treino. Dados: CIFAR-10/100 com ResNet-18.

2.1 Orçamento inicial de cada dispositivo

$$B_e = \left(\alpha_e \frac{|D_e|}{|D|}\right) \cdot (\beta_e R), \qquad \alpha_e,\ \beta_e \sim \mathcal{N}(0.5,\ 0.5) \text{ limitados a } [0.1,\ 1]$$

- $|D_e|/|D|$ é a fração dos dados totais que está no dispositivo e.
- R é o número total de rodadas.
- α_e e β_e são sorteios aleatórios que criam a heterogeneidade de bateria.

2.2 Custo de treino e desconto

O custo por época é proporcional ao tamanho do dataset local e assumido **constante entre rodadas** (os autores dizem que observaram isso, mas não mostram os dados — nota de rodapé 1):

$$b_e = \frac{|D_e|}{|D|} \qquad\qquad B_e \leftarrow B_e - b_e\,\eta_e \quad \text{(a cada época local)}$$

η_e é a fração dos dados locais usada no treino. Quando $B_e \le 0$, o dispositivo sai e não participa mais.

2.3 A ideia do LeanFed

Com FedAvg, um dispositivo aguenta R_e rodadas. O LeanFed reduz a fração de dados usada por época para que a bateria dure todas as R rodadas (λ é a taxa de participação):

$$R_e = \frac{B_e}{b_e L_e} \qquad\qquad \eta_e = \frac{1}{\lambda R} \cdot \frac{B_e}{b_e L_e}, \quad \text{limitado a } [0,\ 1]$$

2.4 O que o modelo deixa de fora

- **Sem unidade física:** não há joules nem mAh; a energia é medida em "fração do dataset × épocas".
- **Sem custo de comunicação:** só o treino local gasta energia.
- **Sem diferença de hardware:** dois dispositivos com o mesmo volume de dados gastam o mesmo; a heterogeneidade está só no orçamento inicial (α_e, β_e).
- **O tamanho do dataset se cancela.** Substituindo B_e em R_e:

$$R_e = \frac{\alpha_e \beta_e R}{L_e}$$

Com L_e = 5 épocas, nenhum dispositivo aguenta mais que R/5 rodadas completas no FedAvg; a duração da bateria depende só dos dois sorteios. (Esta conta não está no artigo.)

- Os autores reconhecem, nas limitações, que o método depende de estimativas precisas do nível de bateria, e deixam a validação em dispositivos reais para trabalho futuro.

2.5 Relevância para o seu trabalho

O modelo é fácil de reproduzir no Flower (um contador de energia por cliente e a fração de dados enviada pelo `config`). Código aberto: github.com/robertomatheuspp/leanFed. Para heterogeneidade de modelos, porém, ele não serve diretamente: o custo depende só do volume de dados, não do tamanho do modelo. É preciso um custo que também cresça com a largura do submodelo (por exemplo, proporcional aos FLOPs), idealmente calibrado com medições reais.

3. Sugestões para o mestrado

3.1 Recomendação principal: largura do submodelo escolhida a cada rodada conforme a bateria

A ideia junta duas linhas que hoje andam quase separadas:

- **HeteroFL e FjORD** definem o tamanho do submodelo uma única vez, por categoria de dispositivo, sem considerar bateria.
- **LeanFed e EAFL** consideram bateria, mas todos os clientes treinam o modelo inteiro.

Proposta: a cada rodada, o servidor escolhe para cada cliente uma largura de modelo (100%, 50%, 25%...), considerando a bateria restante, as rodadas que faltam e quanto cada largura custa. O objetivo é fazer cada dispositivo chegar ao fim do treino, como no LeanFed, mas encolhendo o modelo em vez de cortar dados.

> **Pergunta de pesquisa.** Quando a bateria é escassa, o que é melhor reduzir: **dados** (LeanFed), **épocas** (FedProx) ou **tamanho do modelo** (HeteroFL)? E isso muda com o grau de não-IID? Mesmo que a resposta seja “depende”, mapear quando cada opção vence já é uma contribuição.

Diferença em relação ao DR-FL: ele usa MARL, que é complexo, difícil de reproduzir e com energia simulada. Você pode propor uma política simples, analítica ou gulosa, com modelo de energia calibrado em medições reais. Para um mestrado, uma contribuição simples e bem avaliada costuma valer mais que um método sofisticado e frágil.

3.2 Complementos que fortalecem o trabalho

1. Um modelo de energia realista (o ponto mais fraco do LeanFed). Em vez de custo constante:

$$E_e(p) \approx k_e \cdot \text{FLOPs}(p) \cdot |D_e| \cdot L_e \; + \; c_e \cdot \text{params}(p)$$

- O primeiro termo é a computação: cresce com a largura p, o número de amostras e de épocas.
- O segundo termo é a comunicação: cresce com o número de parâmetros enviados.
- k_e e c_e variam por dispositivo e devem ser calibrados com medições:
  - no setup atual, ler a potência da GPU via NVML (`nvidia-smi` ou `pynvml`) dentro dos containers — não é bateria, mas é energia medida de verdade;
  - mais forte: medir alguns pontos num Raspberry Pi ou Jetson com um medidor USB e usar esses números no simulador.

2. Recarga e traces reais de bateria. LeanFed e EAFL tratam a bateria como algo que só descarrega; na prática, celulares carregam à noite. Há pelo menos um trabalho com traces reais de smartphones, com estado de bateria e carregamento (Yang et al., WWW 2021, sobre heterogeneidade em FL com dados de smartphones em larga escala) — confira se os dados estão disponíveis.

3. Justiça entre clientes. Clientes com pouca bateria recebem modelos menores ou abandonam o treino. Se tiverem dados raros (não-IID), o modelo global fica enviesado contra eles. Medir isso (precisão por cliente, desvio padrão, índice de Jain) é barato e costuma impressionar a banca.

3.3 Baselines, métricas e dados

Item	Escolhas
Baselines	FedAvg, EAFL, LeanFed, HeteroFL com categorias fixas, FjORD (todos simples de implementar no Flower)
Métricas	Precisão em função da energia total gasta (mais informativa que precisão por rodada); rodada em que cada cliente abandona; tempo até a precisão-alvo; justiça entre clientes
Dados	CIFAR-10/100 com partição Dirichlet (já existe no projeto) e FEMNIST, que tem heterogeneidade natural por usuário

3.4 Visão geral do roteiro

Passo	O que é	Esforço
1	Bateria simulada no cliente; baselines FedAvg e LeanFed	1 a 2 semanas
2	Submodelos de largura variável (HeteroFL) e agregação heterogênea	Parte mais delicada
3	Política que escolhe a largura conforme a bateria — contribuição principal	Poucas linhas, muitos experimentos
4	Calibrar o custo de energia com medições reais (NVML, Raspberry Pi, Jetson)	Só troca números de entrada

Os Passos 1 e 2 já dão um primeiro artigo de workshop comparando “reduzir dados” com “reduzir modelo”.

4. Roteiro de implementação

Adaptado ao código atual: Flower 1.30 com a API de `Message`, `train()` em `fedavg_app/task.py` e `FedAvg` em `fedavg_app/server_app.py`. Os trechos de código são esboços para mostrar a ideia, não versões finais.

Passo 1 — Bateria simulada nos clientes

Objetivo: cada cliente tem uma bateria que diminui ao treinar e, quando acaba, ele sai do treinamento. Isso já permite reproduzir FedAvg e LeanFed como baselines.

1.1 Bateria inicial

Como no `compute-speed`: um valor por cliente no `--node-config`, gerado pelo `scripts/gen_compose.py` com um novo argumento `--batteries` (ou sorteado com seed, pela fórmula do LeanFed). Isso deixa o experimento reproduzível e visível no `docker-compose.yml`.

```
--node-config=partition-id=3 num-partitions=10 compute-speed=0.5 \
              battery-init=40.0 energy-per-sample=1.0
```

`energy-per-sample` representa a eficiência energética do hardware. Mantenha-o separado da velocidade: um dispositivo lento não é necessariamente um que gasta muita energia.

1.2 Onde guardar o nível da bateria

O processo do ClientApp não guarda variáveis entre rodadas, mas o `context.state` sim (persiste durante o run inteiro naquele nó):

```
def _battery(context) -> float:
    if "battery" not in context.state:
        init = float(context.node_config["battery-init"])
        context.state["battery"] = MetricRecord({"level": init})
    return context.state["battery"]["level"]
```

1.3 Descontar energia durante o treino

A função `train()` já verifica o `deadline` a cada batch; a bateria funciona do mesmo jeito:

```
# cost_factor = 1.0 por enquanto; o Passo 2 o torna dependente da largura
energy = len(yb) * energy_per_sample * cost_factor
battery -= energy
spent += energy
if battery <= 0:
    completed = False
    break
```

Desconte também a comunicação, por exemplo `c * num_parametros` ao receber e ao enviar o modelo — no Passo 2 isso passa a importar, porque submodelos menores têm menos parâmetros. Decida também se a avaliação gasta energia; o LeanFed ignora, mas é mais honesto cobrar.

1.4 Quando a bateria acaba

Use o mesmo mecanismo do `DEADLINE_EXCEEDED`: crie `BATTERY_DEPLETED = 101` e responda com `Error`. Nas rodadas seguintes, o cliente devolve esse erro imediatamente, sem treinar. Alternativa: enviar o que treinou até ali (parecido com o `partial`). Vale ter as duas opções como configuração.

1.5 Reportar ao servidor

Inclua `battery_before`, `battery_after`, `energy_spent` e `alive` no `_log_metrics` e no `MetricRecord` de resposta. O Passo 3 depende disso: o servidor só sabe a bateria dos clientes pelo que eles reportam.

1.6 Baseline LeanFed

Entra no próprio cliente: calcule $\eta_e = B_e / (\lambda\ R\ b_e\ L_e)$, com R = `num-server-rounds` e λ = `fraction-train`, e treine só com uma fração η_e do `trainloader` (por exemplo, um `Subset` sorteado a cada rodada). Use uma opção `energy-policy = "none" | "leanfed"` no `pyproject.toml`.

> **Pronto quando:** você consegue gerar os gráficos de “clientes vivos por rodada” e de “precisão em função da energia acumulada” para FedAvg e LeanFed, reproduzindo qualitativamente as Figs. 1 e 2 do LeanFed (o FedAvg degrada quando os clientes morrem).

Passo 2 — Submodelos com largura variável (estilo HeteroFL)

Objetivo: cada cliente treina uma fatia de largura p do modelo global (por exemplo $p \in \{1, 1/2, 1/4, 1/8\}$), e o servidor agrega fatias de tamanhos diferentes.

2.1 Modelo parametrizado pela largura

No `task.py`, o `Net` passa a receber `width`:

```
class Net(nn.Module):
    def __init__(self, width: float = 1.0) -> None:
        super().__init__()
        c1, c2, c3, h = (max(1, int(c * width)) for c in (32, 64, 128, 256))
        self.conv1 = nn.Conv2d(3, c1, 3, padding=1)    # entrada fixa: 3 canais
        self.conv2 = nn.Conv2d(c1, c2, 3, padding=1)
        self.conv3 = nn.Conv2d(c2, c3, 3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(c3 * 4 * 4, h)
        self.fc2 = nn.Linear(h, 10)                    # saída fixa: 10 classes
```

A CNN atual facilita: não tem BatchNorm, e o `flatten` organiza os valores canal por canal. Assim, os primeiros `c3` canais correspondem exatamente às primeiras `c3*16` colunas do `fc1`, e o fatiamento fica correto sem truques. A MobileNetV3 tem BatchNorm, convoluções depthwise e blocos SE, o que dá bem mais trabalho. Comece pela CNN; se precisar de um modelo maior, use uma ResNet-18 (a usada no HeteroFL).

2.2 Extrair o submodelo (servidor → cliente)

Para cada tensor, pegue os primeiros índices de cada dimensão:

```
def extract(global_sd: dict, width: float) -> dict:
    shapes = Net(width).state_dict()
    return {k: global_sd[k][tuple(slice(0, s) for s in v.shape)].clone()
            for k, v in shapes.items()}
```

2.3 Agregação heterogênea (cliente → servidor)

Cada posição do tensor global é a média ponderada só dos clientes que a treinaram. As posições que nenhum cliente treinou mantêm o valor anterior.

```
def aggregate_hetero(global_sd: dict, updates: list[tuple[dict, int]]) -> dict:
    new = {}
    for k, g in global_sd.items():
        acc, cnt = torch.zeros_like(g), torch.zeros_like(g)
        for sd, n in updates:
            idx = tuple(slice(0, s) for s in sd[k].shape)
            acc[idx] += n * sd[k]
            cnt[idx] += n
        new[k] = torch.where(cnt > 0, acc / cnt.clamp(min=1e-12), g)
    return new
```

2.4 Integração com o Flower

O `FedAvg` pronto supõe tensores de mesmo formato em todos os clientes. Crie uma subclasse que sobrescreva dois métodos (confira as assinaturas exatas na documentação da 1.30):

- `configure_train`: gera uma `Message` por cliente, cada uma com o seu submodelo (`extract(...)`) e com `config["width"] = p`.
- `aggregate_train`: troca a média simples pela `aggregate_hetero`.

No cliente, o `_setup` passa a ler `msg.content["config"]["width"]` e cria `Net(width)` antes do `load_state_dict`.

2.5 Custo energético por largura

Aqui os Passos 1 e 2 se conectam. Numa convolução, entradas e saídas encolhem juntas, então FLOPs e parâmetros caem com aproximadamente p^2: $p = 1/2$ custa cerca de 1/4. Calcule os FLOPs exatos de cada largura (à mão para a CNN, ou com `fvcore`/`thop`) e use no desconto de energia do Passo 1:

$$\text{cost_factor}(p) = \frac{\text{FLOPs}(p)}{\text{FLOPs}(1)}$$

A comunicação usa a mesma ideia, com params(p).

2.6 Avaliação

Além do modelo completo no servidor (`central_evaluate`), avalie o modelo global fatiado em cada largura p: o cliente fraco também vai usar o modelo pequeno na inferência. O HeteroFL reporta exatamente essa tabela.

2.7 Variantes que mudam só a função `extract`

- **HeteroFL:** sempre os primeiros canais (o que está acima).
- **FedRolex:** uma janela que "rola" a cada rodada, de modo que todos os canais acabam treinados.
- **FjORD:** o cliente sorteia $p' \leq p$ a cada batch (ordered dropout).

> **Pronto quando:** um teste unitário confirma que, com todos os clientes em $p = 1$, `extract` + `aggregate_hetero` dá resultado idêntico ao FedAvg; e, com categorias fixas (1/3 dos clientes com $p = 1$, 1/3 com 1/2 e 1/3 com 1/4) e o mesmo orçamento de bateria, você compara FedAvg, LeanFed e HeteroFL. Esse é o experimento "reduzir dados vs. reduzir modelo", que já pode virar o artigo de workshop.

Passo 3 — Política de escolha da largura conforme a bateria

Objetivo: em vez de categorias fixas, o servidor escolhe p_e a cada rodada, com base na bateria restante e nas rodadas que faltam. É a contribuição principal da dissertação.

3.1 O que o servidor precisa saber de cada cliente

Informação	De onde vem
Bateria atual B_e	Último `battery_after` reportado (Passo 1.5)
Eficiência k_e (energia por FLOP·amostra)	Estimada pelos relatórios: `energy_spent / (FLOPs(p) · amostras)`
Tamanho do dataset local n_e	`num_examples`, que já é reportado
Rodadas restantes	`num_rounds − server_round + 1`

Na primeira rodada o servidor ainda não sabe nada. Duas soluções:

- uma rodada de sondagem antes do treino, com uma mensagem de query para os clientes (o ClientApp aceita um handler para isso; confira na 1.30);
- começar de forma conservadora, com p pequeno.

3.2 Custo previsto por largura

$$\hat{E}_e(p) = k_e \cdot \text{FLOPs}(p) \cdot n_e \cdot L_e \; + \; c \cdot \text{params}(p)$$

3.3 Política base: dividir a bateria igualmente entre as rodadas restantes

```
WIDTHS = (1.0, 0.5, 0.25, 0.125)

def choose_width(battery: float, rounds_left: int, cost: dict[float, float]) -> float:
    budget = battery / rounds_left
    for p in WIDTHS:                  # da maior para a menor
        if cost[p] <= budget:
            return p
    return 0.0                        # não participa nesta rodada
```

Propriedade útil: quando o cliente pula uma rodada ($p = 0$), a energia não gasta aumenta o orçamento das rodadas seguintes. Assim, ele participa de forma intermitente em vez de morrer cedo. É simples, interpretável e fácil de defender numa banca.

3.4 Variações a comparar (o que transforma a política em pesquisa)

1. **Qual mecanismo reduzir:** só a largura (a proposta), só os dados (LeanFed), só as épocas, ou uma combinação (reduzir largura até um p mínimo e depois cortar dados). É a pergunta central do trabalho.
2. **Como distribuir o orçamento no tempo:** igual em todas as rodadas, mais nas primeiras ou mais nas últimas. O LeanFed sugere que estar vivo no final importa; teste se compensa guardar energia para o fim.
3. **Margem de segurança:** usar $B_e \cdot (1 - m)$ para tolerar erro na estimativa de k_e. Os autores do LeanFed admitem que dependem de estimativas exatas da bateria.

4. **Seleção de clientes junto com a largura:** quando sobram muitos clientes, priorizar quem tem mais bateria, como no EAFL (outro baseline).

3.5 Integração com o Flower

Tudo fica na mesma subclasse do Passo 2:

- `aggregate_train`: além de agregar, atualiza `self.clients[node_id] = {battery, k, n}` com as métricas recebidas (o `node_id` vem de `reply.metadata.src_node_id`).
- `configure_train`: consulta esse dicionário, chama `choose_width` para cada nó e monta as mensagens. Clientes com $p = 0$ não recebem mensagem.

3.6 Avaliação final

- Precisão final do modelo completo e de cada largura.
- Precisão em função da energia total gasta (a métrica principal).
- Rodada em que cada cliente morre (boxplot como a Fig. 2 do LeanFed).
- Justiça: precisão por cliente no seu conjunto de validação (desvio padrão e índice de Jain), principalmente com Dirichlet α baixo.

Passo 4 e ordem prática

O Passo 4 (calibrar k_e e c com medições reais, via NVML ou Raspberry Pi/Jetson) só troca números de entrada; por isso pode vir por último sem atrasar o resto. A ordem sugerida é 1 → 2 → 3 → 4, com o artigo de workshop ao final do Passo 2.

5. Referências

[1] Xia & Shi (2024). Towards Energy-Aware Federated Learning via MARL: A Dual-Selection Approach for Model and Client (DR-FL). https://arxiv.org/abs/2405.08183

[2] Zhang et al. Seleção adaptativa de modelos para agentes móveis com bateria limitada. IEEE TCCN. https://cis.temple.edu/~wu/research/publications/Publication_files/TCCN-AAI-26-0050.R1-finalPDF.pdf

[3] Pfeiffer et al. (2024). Energy-Aware Heterogeneous Federated Learning via Approximate Systolic DNN Accelerators. https://arxiv.org/abs/2402.18569

[4] Arouj & Abdelmoniem (2022). Towards Energy-Aware Federated Learning on Battery-Powered Clients (EAFL). https://arxiv.org/abs/2208.04505

[5] Kim & Wu (2021). AutoFL: Enabling Heterogeneity-Aware Energy Efficient Federated Learning. MICRO. https://arxiv.org/abs/2107.08147

[6] Pereira, Vaca-Rubio & Blanco. Learn More by Using Less: Distributed Learning with Energy-Constrained Devices (LeanFed). https://arxiv.org/abs/2412.02289

[7] Código do LeanFed. https://github.com/robertomatheuspp/leanFed

[8] Exploring System-Heterogeneous Federated Learning with Dynamic Model Selection. https://arxiv.org/abs/2409.08858

[9] Citados de memória (sem link verificado): HeteroFL (ICLR 2021), FjORD (NeurIPS 2021), FedRolex (NeurIPS 2022), ScaleFL (CVPR 2023), DepthFL (ICLR 2023), Yang et al. (WWW 2021).